

Chapter 04 공개키 암호(3)

목차

4.1 개요

4.2 배낭암호

4.3 RSA

4.4 디피-헬먼

4.5 타원곡선 암호

4.6 공개키 표기법

4.7 공개키 암호 용도

4.8 인증서와 PKI

4.9 양자 컴퓨터와 공개키

4.10 요약

4.6 공개키 표기법

- 앨리스의 공개키로 메시지 M 을 암호화: $C=\{M\}_{\text{앨리스}}$
- 앨리스의 개인키로 암호문 C 를 복호화: $M=[C]_{\text{앨리스}}$
- 앨리스가 메시지 M 에 한 서명: $S=[M]_{\text{앨리스}}$
 - 중괄호는 공개키 연산, 대괄호는 개인키 연산, 아래첨자 이름은 누구의 키를 사용하는 것인지 보여줌
- 공개키와 개인키 연산은 역으로 함
 - 공개키는 항상 공개 → 누구나 $\{M\}_{\text{앨리스}}$ 를 계산 가능
 - 개인키는 비밀 → 오직 앨리스만이 $[C]_{\text{앨리스}}$ 또는 $[M]_{\text{앨리스}}$ 를 계산

4.7 공개키 암호 용도

➤ 공개키 암호의 장점

- 공개키 암호는 송/수신자 사이에 사전에 비밀 공유키를 설정할 필요가 없음
- 무결성 뿐만 아니라 부인봉쇄(non-repudiation) 제공

➤ 현실 세계에서의 보안성

- 효율적이면서 공유 비밀키를 미리 나누어 가질 필요가 없는 것이 가능할까?
↳ 방법은 **합성(Hybrid) 암호체계**를 이용하는 것

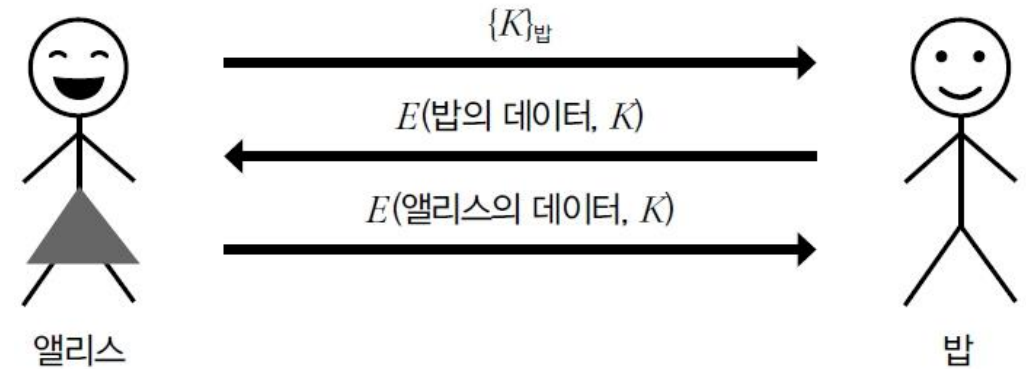


그림 4-5 합성 암호체계

4.7 공개키 암호 용도

➤ 효율적이면서 공유 비밀키가 없는 송/수신자 사이의 암호화 통신이 가능한가?

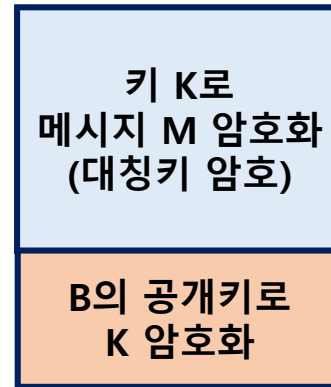


Alice

- (1) 메시지 암호화에 사용할 키 K 생성
- (2) Bob의 공개키를 이용하여 K 암호화

- (3) 메시지 M은 $\{K\}_{Bob}$ 를 이용하여 대칭키 암호 방식으로 암호화

$$C = E(\text{메시지 } M, K)$$



Bob

- (4) 자신의 개인키로 K 복호화

- (5) K를 이용하여 메시지 K 복호화

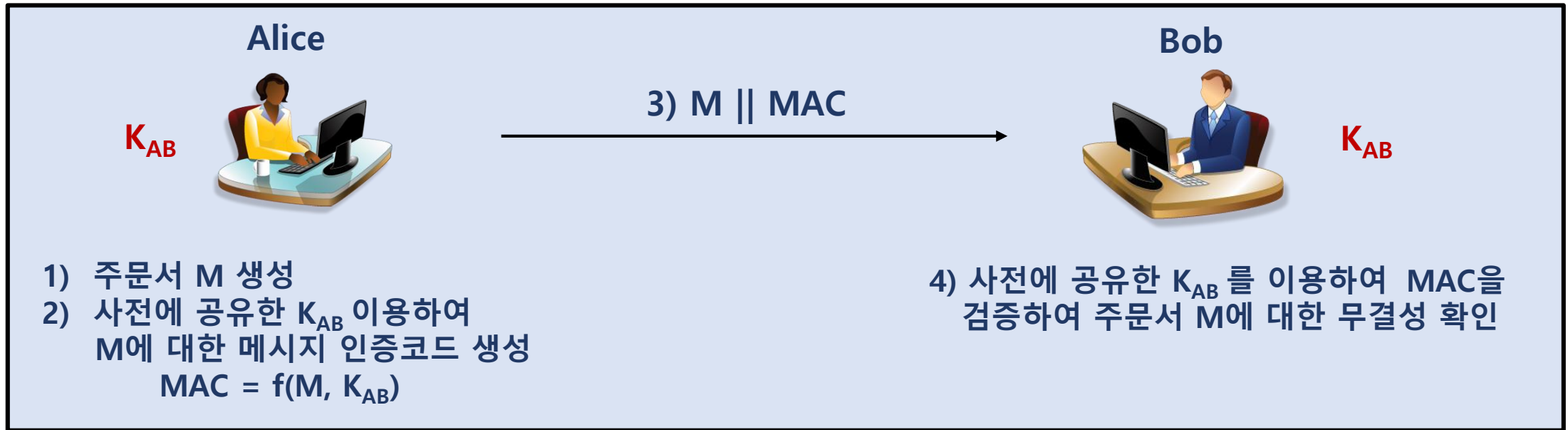
$$M = D(\text{암호문 } C, K)$$

[하이브리드 암호 시스템]

4.7 공개키 암호 용도

➤ 대칭키 암호 방식을 이용한 무결성 확인 예

- Alice가 증권 중개인 Bob에게 주식 100주를 주문하는 경우
 - 주문의 무결성을 확보하기 위해 **사전에 공유한 키 K_{AB} 를 이용하여 메시지 인증코드(MAC) 계산하여 주문서 M과 함께 전송**
 - Bob은 **MAC을 검증하여 주문서가 변경되지 않고 도착하였음을 확인한 후 주문 처리**



4.7 공개키 암호 용도

➤ 대칭키 암호 방식을 이용한 무결성 확인 예

- Alice가 주문을 넣은지 얼마 지나지 않아 Bob에게 돈을 보내기 전에 주식의 가치가 급락
- 이 시점에서 Alice가 주문서를 보낸 사실을 부인하는 경우 **Bob은 Alice가 주문을 넣었다는 것을 증명할 수 있을까?**
 - Bob이 가진 것이 MAC밖에 없다면 증명할 방법이 없음
 - MAC은 Alice와 Bob이 공유한 키 K_{AB} 를 이용하여 생성하였으므로, **Alice 뿐만 아니라 Bob도 생성할 수 있음**
 - 제 3자에게 주문서를 작성한 사람이 Alice 라는 것을 증명하기 위해서는, **Alice만이 알고 있는 정보**를 이용하여 생성한 정보가 필요
 - **공개키 암호 시스템의 개인키를 이용하는 방식 필요(전자서명)**

4.7 공개키 암호 용도 – 서명과 부인봉쇄

➤ 전자 서명(Digital Signature)

- 종이문서의 수기 서명과 같은 역할을 하는 암호학적 메커니즘
- 메시지가 생성된 이후 변경되지 않았음을 보장하는 **메시지 인증**과 메시지를 생성한 사용자가 누구인지 **사용자 인증**을 동시에 제공

➤ 전자 서명의 종류

- RSA 서명
- ElGamal 서명
- DSA(Digital Signature Algorithm, 미국 표준)/ECDSA(Elliptic Curve DSA)
- KCDSA(Korean Certificate based Digital Signature Algorithm, 한국 표준)

[참고] 보안 서비스

➤ 대표적인 보안 서비스

- 기밀성 : 인가된 사람 이외에는 정보에 접근할 수 없음
- 무결성 : 메시지가 권한이 없는 사람에 의해 변경되지 않았음을 보장
- 데이터 출처 인증 : 메시지를 생성한 사람의 신원 확인
- 부인 봉쇄(방지) : 메시지를 생성한 후 부인할 수 없음을 보장

➤ 응용 환경마다 서로 다른 보안 서비스를 필요로 함

- 개인 이메일 : 기밀성, 무결성, 데이터 출처 인증
- 기업 이메일 : 기밀성, 무결성, 데이터 출처 인증, 부인방지
- 휴대폰 소프트웨어 업데이트 : 코드 무결성

➤ 기밀성은 주로 대칭키 암호 시스템으로 제공하고 무결성, 데이터 출처인증, 부인 방지는 전자서명으로 제공 가능

[참고] 보안 서비스

➤ 추가적인 보안 서비스

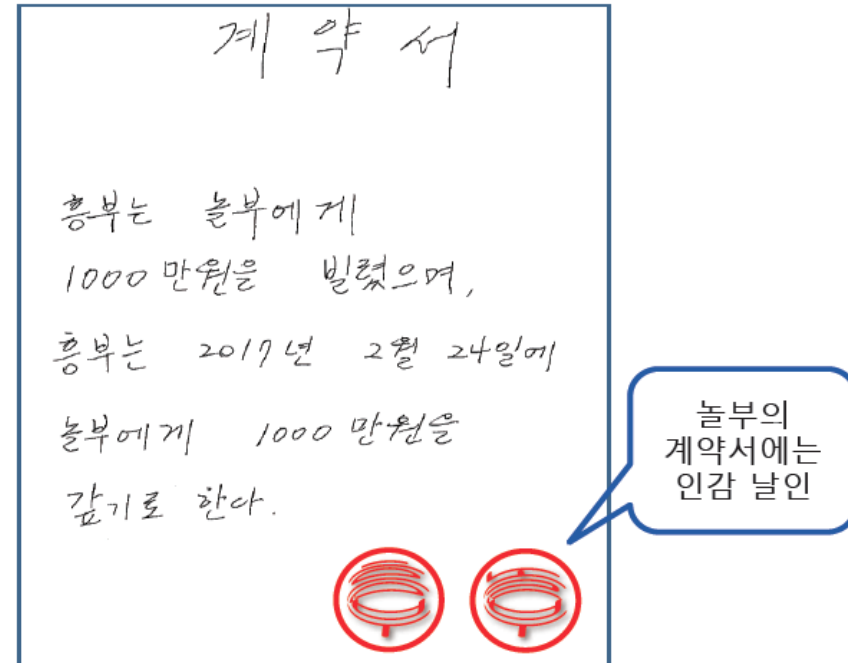
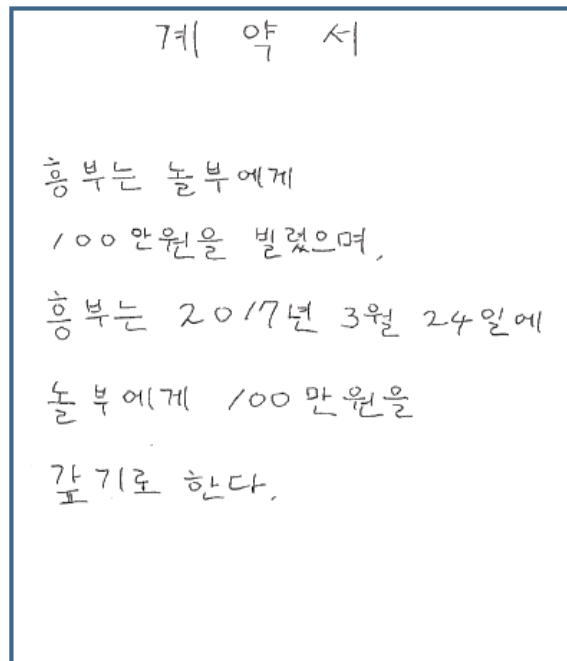
- **개인식별/객체 인증** : 객체(사람, 컴퓨터, 디바이스 등)의 신원 확인
- **접근 통제** : 권한을 가진 객체만이 자원에 접근하도록 제한
- **가용성** : 권한이 있는 사용자는 원하는 서비스를 받을 수 있음
- **감사** : 특정 이벤트에 대한 로그를 관리 하는 등 보안관련 활동 수행에 대한 증거 제공
- **물리적 보안** : 물리적 변경이나 시도에 대한 보호대책 제공
- **익명성** : 신원 노출이나 남용에 대한 보안 대책 제공

➤ 시스템에서 고려되어야 하는 보안 서비스는 응용 서비스에 따라 특화됨

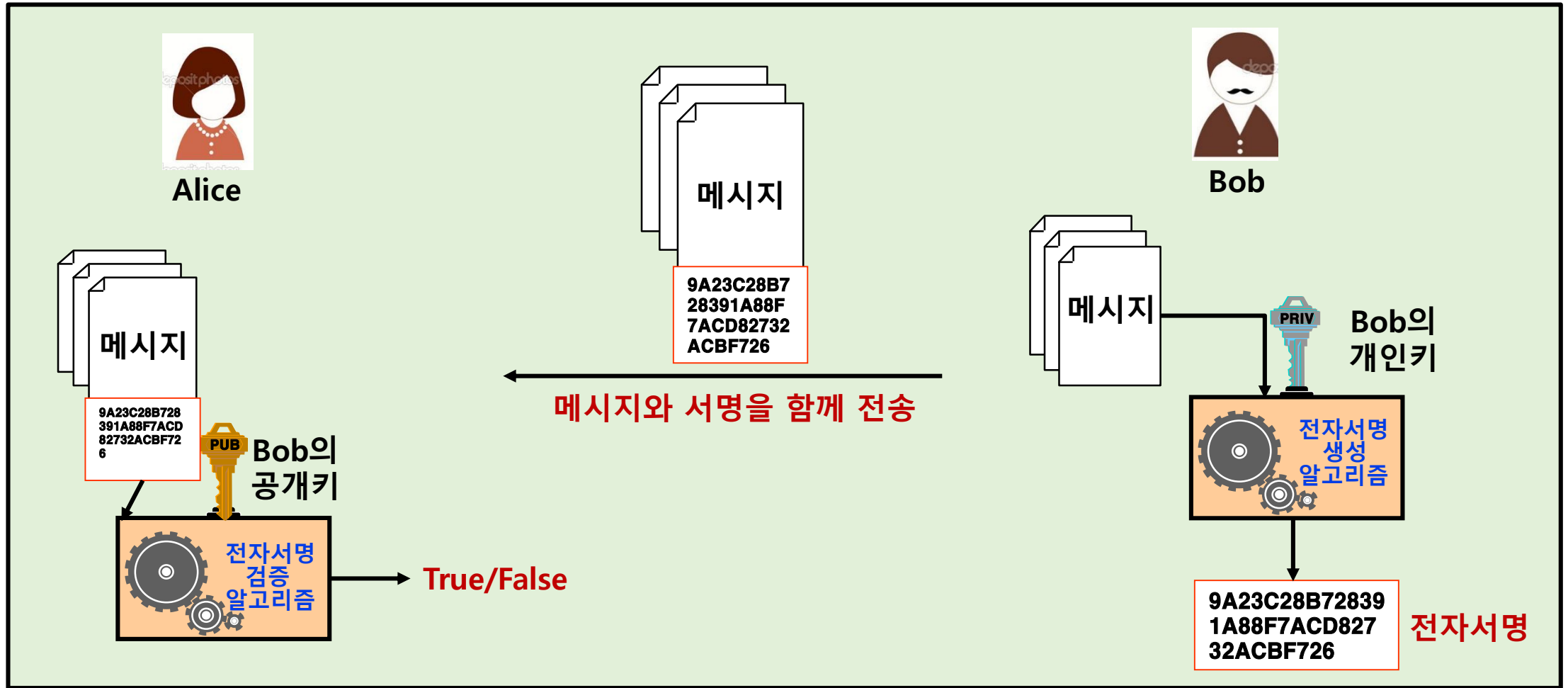
- 이메일에서 익명성은 보장되지 않음
- 자동차 간 통신 시스템에서는 익명성 보장 필요
- 운영체제 보안을 위해서는 시스템에 대한 접근 통제 필요

전자서명의 원리

- 특정 사람이 문서를 생성했다는 것은 현실 세계에서든 매우 중요
 - 도장이나 서명이 포함된 문서는 법적으로 인정받을 수 있음



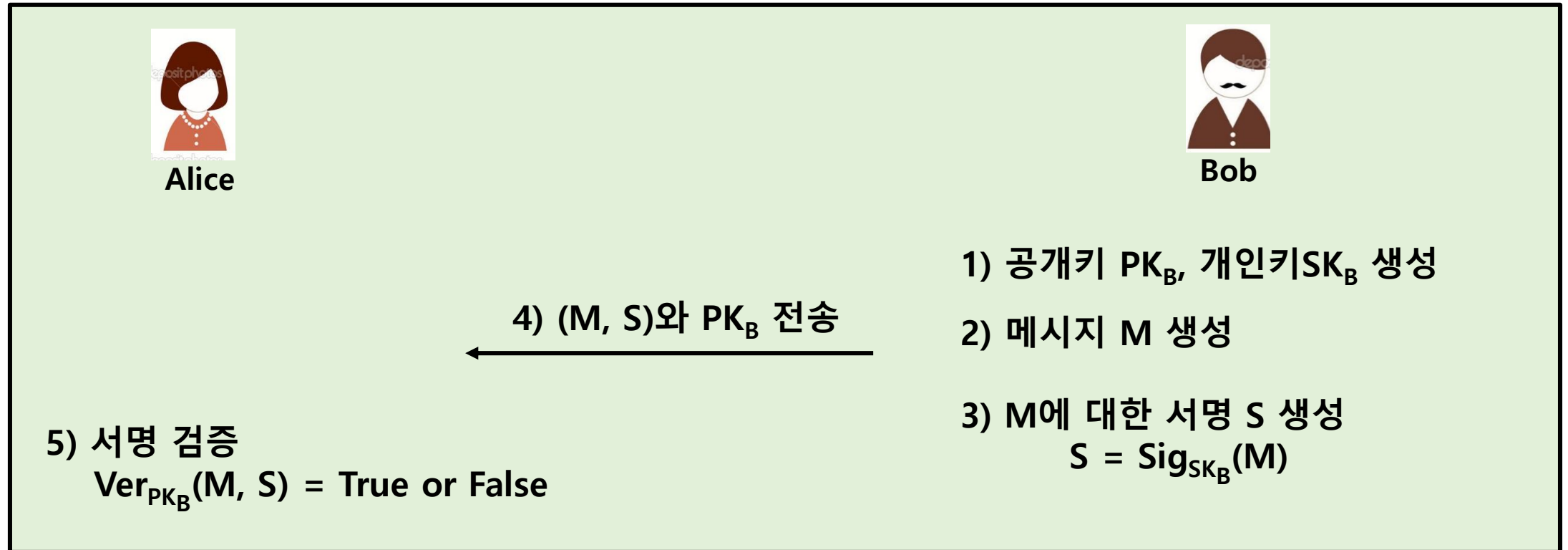
전자서명의 원리



[전자서명 생성 및 검증 과정]

전자서명의 원리

- 메시지와 서명은 함께 전송되어야 함
- 메시지는 평문으로 전송되므로 기밀성은 보장하지 않음



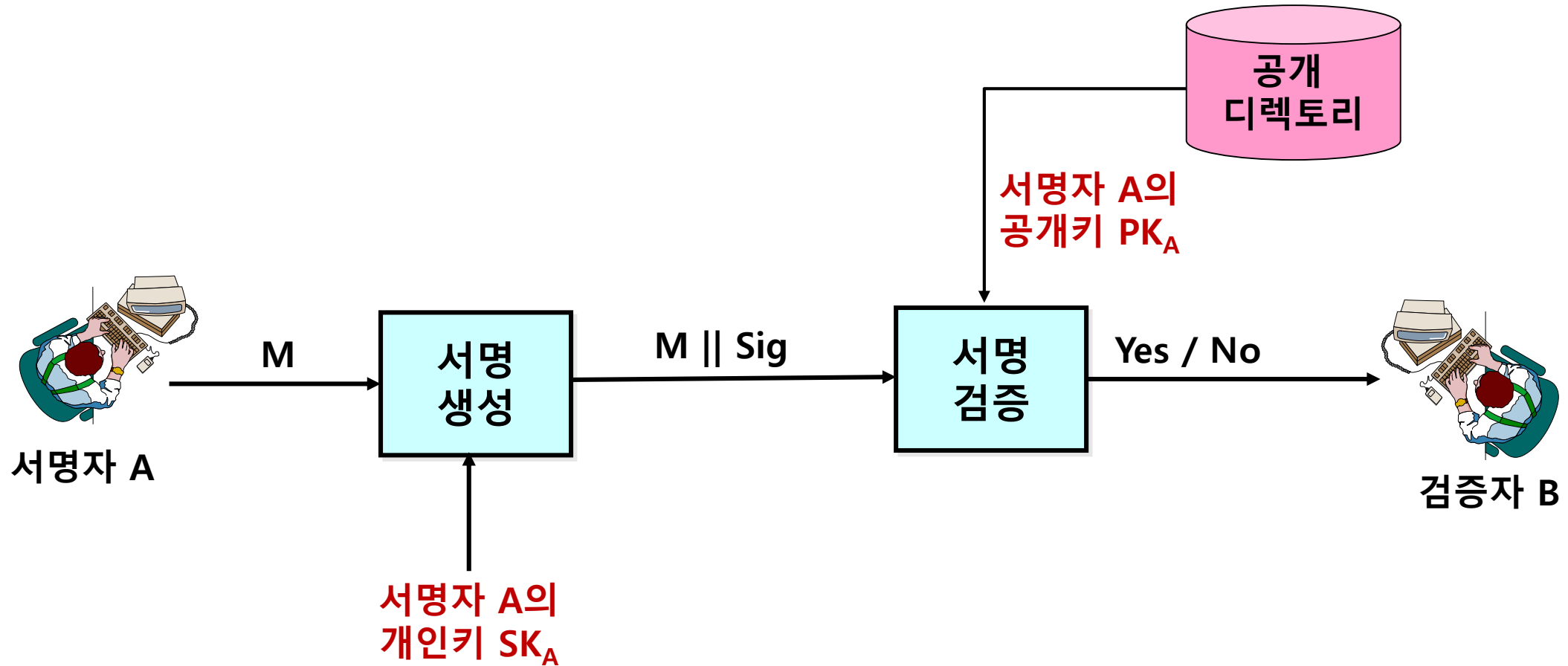
[전자서명 생성 및 검증]

전자 서명이 제공하는 보안 기능

- 위조불가 (unforgeable)
 - 서명자 외에 다른 사람은 해당 서명자의 서명을 위조할 수 없음
- 서명자 인증 (user authentication)
 - 누구든지 전자서명을 생성한 사람을 확인할 수 있음
- 부인불가 (non-repudiation)
 - 서명자는 서명한 후에 자신이 서명한 사실을 부인할 수 없음
- 변경불가 (unalterable)
 - 서명한 후에 메시지의 내용을 변경할 수 없음
- 재사용 불가 (not reusable)
 - 하나의 문서의 서명을 다른 문서의 서명으로 재사용할 수 없음

전자 서명

➤ 공개키 암호 방식을 이용한 전자 서명



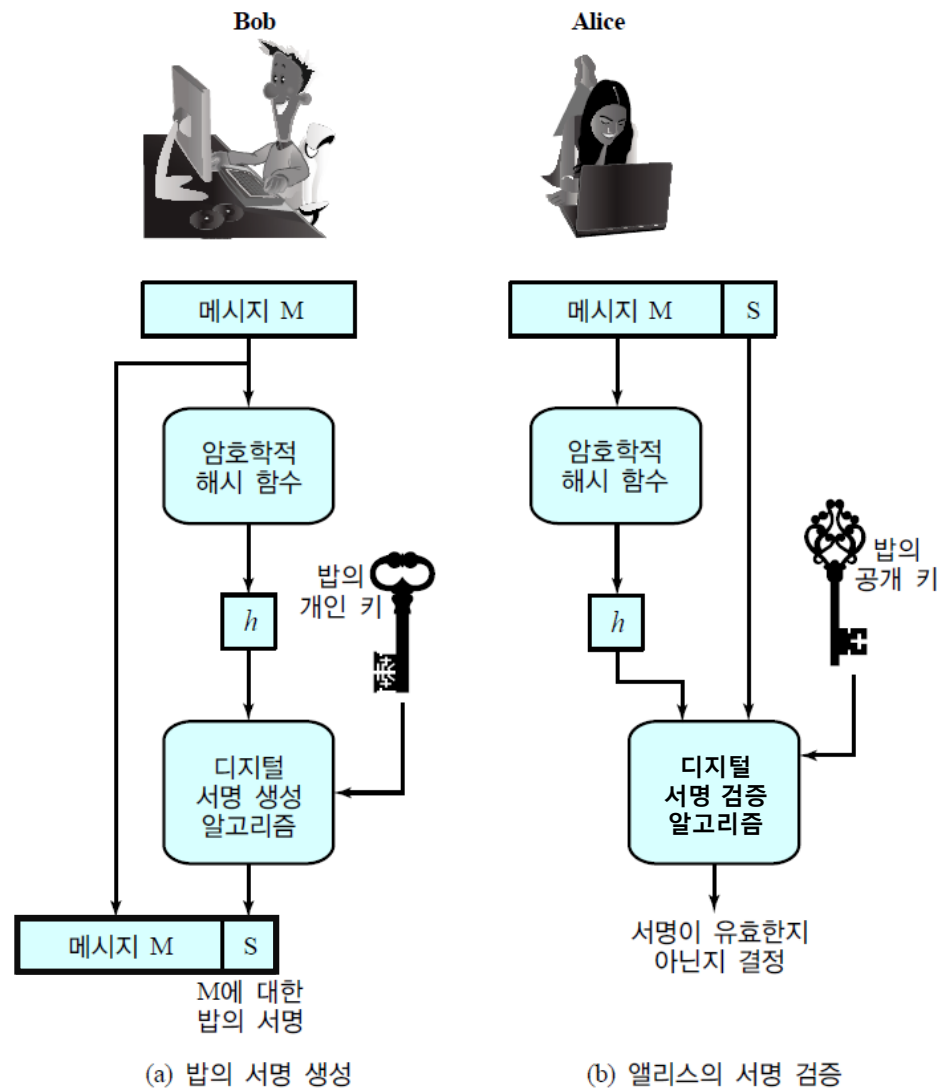
전자서명

➤ 서명 생성의 입력

- 메시지 M
- 서명자의 개인키

➤ 서명 검증의 입력

- 메시지 M
- 서명 S
- 서명자의 공개키



RSA 전자 서명 방식

- p, q : 소수
- $n = p \times q$
- $\Phi(n) = (p-1)(q-1)$
- A의 공개키 $e_A : (e_A, \Phi(n))=1$
- A의 개인키 $d_A : e_A \cdot d_A \equiv 1 \pmod{\Phi(n)}$



서명자 A

M

$$S = M^{d_A} \pmod{n}$$

개인키 d_A

$M \parallel S$

$$M \stackrel{?}{=} S^{e_A} \pmod{n}$$

Yes / No



검증자 B

공개
디렉토리

공개키 e_A

RSA 전자 서명 방식

[RSA 전자서명 키]

- 서명자 A의 공개키 : (n, e)
- 서명자 A의 개인키 : d

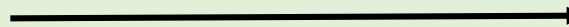
[RSA 전자서명 프로토콜]



Alice

- 2) 메시지 M 생성
- 3) 서명 $S \equiv M^d \pmod n$ 계산

1) 공개키 (n, e) 전송



Bob

4) (M, S) 전송



5) 서명 검증

$M' \equiv S^e \pmod n$ 계산
[$M=M'$ 이면, Alice의 유효한 서명
 $M \neq M'$ 이면, Alice의 유효한
서명이 아님]

RSA 전자 서명 방식

$M' \equiv S^e \pmod n$ 계산
[$M=M'$ 이면, Alice의 유효한 서명
 $M \neq M'$ 이면, Alice의 유효한 서명이 아님

➤ $M=M'$ 인 경우, 다음이 성립

$$M' \equiv S^e \equiv (M^d)^e \equiv M \pmod n$$

- 서명 생성에 사용된 **개인키 d**와 **공개키 e**가 한 쌍의 **RSA 키 쌍**이라는 것을 의미
- 공개키 e에 대응하는 개인키 d는 Alice만이 알고 있으므로 Alice의 서명임을 확인할 수 있음 (Alice가 메시지를 생성했음을 의미) -> **사용자 인증**
- 또한, $M=M'$ 라는 것은 메시지가 전송 중에 변경되지 않았음을 의미 -> **메시지 인증**
- 전자서명이 유효한 경우, **사용자 인증**과 **메시지 인증**을 동시에 제공

RSA 전자 서명 방식

[예제] Bob이 메시지(M=4)와 이에 대한 서명을 Alice에게 전송하고자 한다.



Alice



Bob

6) 공개키 $(n, e) = (33, 3)$



8) $(M, S) = (4, 16)$



9) 서명 검증

$$M' = 16^3 \bmod 33 \equiv 4 \bmod 33$$

$M' = M$ 이면, 유효한 서명

- 1) $p=3, q=11$ 선택
- 2) $n = pq = 33$
- 3) $\Phi(n) = (3-1)(11-1) = 20$
- 4) 공개키 $e=3$ 선택
- 5) 개인키 $d \equiv e^{-1} \bmod 20 \equiv 7 \bmod 20$ 계산

7) 서명 생성

$$M=4$$

$$S = M^d \equiv 4^7 \bmod 33 \equiv 16 \bmod 33$$

RSA 전자서명 방식

- RSA 서명은 모듈러 n 만큼의 길이이며, n 은 일반적으로 1024비트에서 3072비트임
- 서명의 생성과 검증은 위해 모듈러 역승이 필요하며, 제공 후 곱셈 알고리즘 이용
- 서명 생성은 한번만 이루어지지만, 서명 검증은 여러 번 이루어짐
 - 검증을 효율적으로 하기 위해 짧은 공개키 사용
 - 단, 개인키는 안전성을 제공할 수 있도록 충분히 긴 길이를 사용해야 함
- RSA 서명 방식의 안전성은 소인수분해 문제에 기반

➤ 시스템 구성

- p : 큰 소수(1024 bit 이상)
- $p-1$ 의 약수인 소수 q (160bit 이상) 생성
- 위수가 q 인 원소 α 선택

※ p, q, α 는 공개 파라미터

- 개인키 선택 : 임의의 정수 $d \in \{1, 2, 3, \dots, q-1\}$ 선택
- 공개키 계산 : $\beta \equiv \alpha^d \pmod p$ 계산

- DSA는 큰 소수 p 와 $p-1$ 의 약수인 소수 q 사용

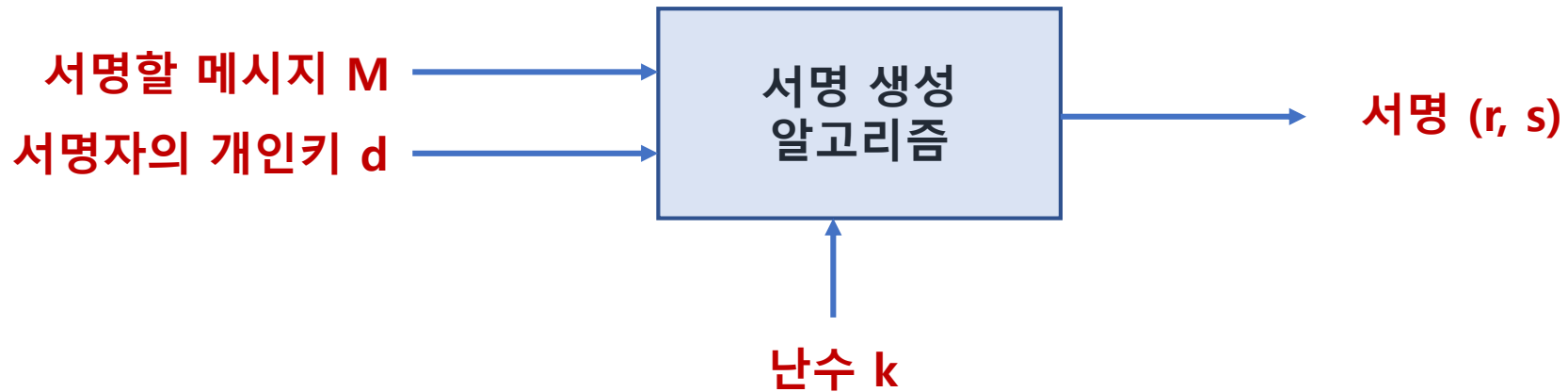
[DSA의 주요 파라미터의 길이(비트)]

p	q	서명 길이
1024	160	320
2048	224	448
3072	256	512

DSA 서명 방식

➤ DSA 서명 생성(입력: 서명할 메시지 M, 개인키 d, 공통 파라미터 α , p, q)

- q보다 작은 랜덤 수 k 선택
- 서명 계산
$$r \equiv (\alpha^k \bmod p) \bmod q$$
$$s \equiv (M + d \cdot r)k^{-1} \bmod q$$

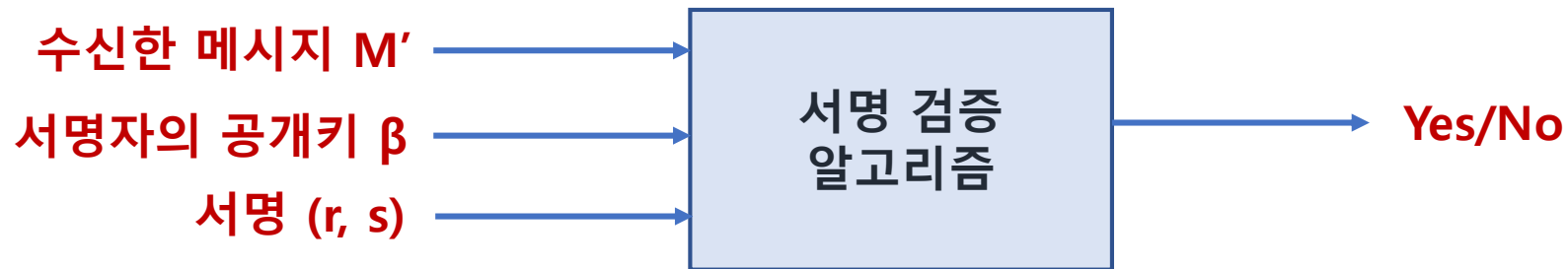


DSA 서명 방식

- DSA 서명 검증(입력 : 수신한 메시지 M' , 서명(r, s), 공개키 β , 공통 파라미터 α, p, q)

- 다음의 검증식이 만족하는지 확인

$$r \equiv \beta^{s^{-1}r} \alpha^{s^{-1}M'}$$



DSA 서명 방식

➤ 서명 검증 과정

$$\begin{aligned} r &\equiv \beta^{s^{-1}r} \alpha^{s^{-1}M'} \pmod{q} \quad (\beta \equiv \alpha^d \pmod{p} \text{ 이므로}) \\ &\equiv \alpha^{ds^{-1}r} \alpha^{s^{-1}M'} \pmod{q} \\ &\equiv (\alpha^d)^{\frac{k \cdot r}{M + d \cdot r}} \alpha^{\frac{k \cdot M'}{M + d \cdot r}} \pmod{q} \end{aligned}$$

$$\begin{aligned} r &\equiv (\alpha^k \pmod{p}) \pmod{q} \\ s &\equiv (M + d \cdot r)k^{-1} \pmod{q} \end{aligned}$$

$$\begin{aligned} &\equiv \alpha^{\frac{K(M' + d \cdot r)}{M + d \cdot r}} \pmod{q} \quad (\text{서명 생성에 사용한 } M \text{과 수신한 } M' \text{가 같은 경우}) \\ &\equiv \alpha^k \pmod{q} \end{aligned}$$

[검증식이 만족하는 경우 다음 2가지 사실을 확인할 수 있음]

- 서명 생성에 사용한 메시지 M과 수신한 메시지 M'가 동일 → M이 변경되지 않음(메시지 인증)
- 공개키 β 를 이용하여 서명을 검증하였으므로, 서명 생성에 쓰인 개인키 d는 검증에 사용한 공개키 β 를 생성하는데 사용한 d임을 확인 → 개인키 d를 알고있는 사람이 서명을 생성함(사용자 인증)

DSA 서명 방식

➤ 연산량

- 서명을 생성하기 위해 r, s 를 계산해야 함
 - $r \equiv (\alpha^k \bmod p) \bmod q$ 을 계산하는데 사용하는 k 는 q 보다 작은 수이므로, 160비트임
 - 제곱 후 곱셈 알고리즘을 사용하는 경우 약 $1.5 \times 160 = 240$ 개의 제곱과 곱셈 수행
 - $s \equiv (M + d \cdot r)k^{-1} \bmod q$ 를 계산하기 위해서는 대부분의 연산이 k 의 역원을 구하는데 사용

➤ 안전성

- 안전성은 이산대수 문제에 의존
- 원시원소 대신 위수가 q 인 원소 α 를 사용하므로, 공개키로 부터 개인키가 노출되지 않기 위해 p 가 1024비트인 경우 q 는 160 이상 되어야 함

[DSA의 표준 파라미터의 비트 길이 및 보안 수준]

p(비트)	q(비트)	대칭키 암호방식의 키 길이(비트)
1024	160	80
2048	224	112
3072	256	128

➤ KCDSA(Korean Certificate-based Digital Signature Algorithm)

- 국내 표준 전자서명 방식
- 인증서 기반 부가형 전자 서명 알고리즘
- 1998년 채택, 2000년 개정안 발표

➤ 시스템 구성

- p : 소수, $2^{\alpha-1} < p < 2^\alpha$ ($|p| = \alpha$), $\alpha = 1024 + 256i$ ($0 \leq i \leq 4$)
- q : 소수, $q | p-1$ 이고 $2^{\beta-1} < q < 2^\beta$ ($|q| = \beta$), $\beta = 160 + 32j$ ($0 \leq j \leq 3$)
- g : Z_q 의 원시 원소, $g^q = 1 \bmod p$
- x_A : Z_q 상의 원소, 사용자 A의 개인키
- y_A : $y_A = g^{x_A} \bmod p$, 사용자 A의 공개키
- $h()$: 출력이 $|q|$ 비트 이하인 해시 함수

KCDSA 서명 생성

➤ 서명 생성 과정 : 서명 생성자 A는 자신의 개인키 x_A 를 이용하여 다음과 같이 메시지 M에 대한 전자 서명 생성

- ① 랜덤 수 $k \in_R Z_q$ 를 선택하고, $W \equiv g^k \bmod p$ 를 계산한다.
- ② 해시 함수를 이용하여 중간 값 $R = h(W)$ 를 계산한다.
- ③ $Z = y_A \bmod 2^l$ 를 구하고, 서명할 메시지 M과 Z에 대한 해시 값 $H = h(Z||M)$ 를 계산한다. (l : 해시 함수블록의 크기)
- ④ $E \equiv (R \oplus H) \bmod q$ 를 계산한다.
- ⑤ 개인키 x_A 를 이용하여 서명 $S \equiv x_A(k - E) \bmod q$ 를 계산한다.
- ⑥ 메시지 M과 서명 (R, S)를 수신자에게 전송한다.

KCDSA 서명 검증

➤ 서명 검증 과정 : 서명 검증자 B는 A의 공개키 y_A 를 이용하여 메시지 M에 대한 서명 (R, S)를 다음과 같이 검증한다.

- ① 서명자의 공개키를 이용하여 $Z = y_A \bmod 2^l$ 를 계산한다.
- ② 수신한 메시지 M과 ①의 Z를 이용하여 $H' = h(Z||M)$ 를 계산한다.
- ③ 수신한 R과 ②의 H' 를 이용하여 $E' \equiv (R \oplus H') \bmod q$ 를 계산한다.
- ④ 서명자의 공개키와 ③의 E' 를 이용하여 $W' \equiv y_A^S g^{E'} \bmod p$ 를 계산한다.
- ⑤ $H(W') = R$ 을 만족하는지 확인한다.

KCDSA 서명 검증 과정

$$W' \equiv y_A^S g^{E'} \bmod p$$

$$\equiv (g^{x_A^{-1}})^S g^{E'} \bmod p$$

$$\equiv (g^{x_A^{-1}})^{x_A(k-E)} g^{E'} \bmod p$$

$$\equiv g^{(k-E)} g^{E'} \bmod p$$

$$\equiv g^k \bmod p \text{ (E와 E'가 동일한 경우에 W'와 W값이 동일)}$$

$$S \equiv X_A(k - E) \bmod q$$

$H(W') = R$ 을 만족하는지 확인

=> 만족하는 경우, 서명에 사용된 개인키와 y_A 생성에 사용한 개인이 동일하고, 서명에 사용한 메시지와 수신한 메시지가 동일함을 확인

정리

- 공개키 암호 방식은 송/수신자 사이에 사전에 비밀 공유키를 설정할 필요가 없음 → 암호문을 보내기 위해서는 수신자의 공개키만 필요
- 사전에 공유한 비밀키가 없는 송/수신자 사이에서 효율적으로 암호 통신을 하기 위해서는 **하이브리드 암호 체계 이용** → 메시지 암호화에는 대칭키 암호 방식을 이용하고, 대칭키 암호화에 사용한 키는 공개키 암호 방식을 이용하여 암호화 수
- 공개키 암호 방식은 사용자 A만이 알고있는 정보인 개인키가 존재하므로 이를 이용하여 **무결성 뿐만 아니라 부인봉쇄(non-repudiation)** 제공 가능
- 서명자의 개인키를 이용하여 전자서명을 생성하고, 서명자의 공개키를 이용하여 전자서명을 검증하는 경우 **메시지 무결성, 사용자 인증 및 부인 불가** 서비스 제공
- RSA는 가장 널리 사용되는 전자서명 알고리즘이며, 미국 표준방식인 DSA 방식과 국내 표준 방식인 KCDSA는 이산대수 문제를 이용한 전자서명 방식임
- RSA와 DSA 모두 최소 1024비트 이상이어야 하며, 장기적인 보안을 위해서는 3072비트의 모듈러 값을 사용할 것을 권장